WELCOME

COS 201 LECTURE 1
PRISCILLA OYEBADE

Course Information Highlights

- **Core Concepts**
  - Understand **what computation is** and how computers solve problems.
  - Learn **Python programming basics**, including mathematical operations.
  - Work with **Python variables and data types** to store and manipulate data.
  - Gain practical experience in writing, testing, and debugging small programs.

By mastering these topics, you will build a strong foundation for future programming and computer science courses.

- **Coding Sessions: 30 minutes is dedicated to coding** in each class.
- **Class Experience:** It is advisable to join the class with a **PC for a better coding experience**.

Three Basic Outcomes

Upon completing this class, you will gain three basic things:

- **Problem Solving**
- **Knowledge of Concepts**
- **Programming Skill**

Learning Objectives

Throughout this course, you will:

- Learn the **fundamentals of computer programming**.
- Develop **critical thinking and problem-solving skills**.
- **Think creatively** to solve problems.
- Express solutions **clearly and accurately**.
- Learn to implement **non-trivial programs** in a high-level programming language.
- Develop **programming maturity** (confidence, willingness to experiment).

LET’S START

What is knowledge?(Let’s interact)

- We need to learn the nature of knowledge because it forms the fundamental conceptual framework for **computation** and **programming**.
- Understanding the difference between the two types of Knowledge: **Declarative** and **Imperative** knowledge. It is essential because programming is the act of translating one into the other.

The Nature of Knowledge: Declarative vs. Imperative

Declarative knowledge is statements of fact (What we Know)

Imperative knowledge is a recipe or “how-to”

Declarative simply means **expressing or stating something as a fact**.

In learning or knowledge, **declarative knowledge** refers to:

“Knowing *what* something is.”

It includes facts, definitions, and information you can **declare**, speak, or write.

Examples (general):

- Knowing that the sky is blue
- Knowing the meaning of a term
- Knowing that 2 + 2 = 4

So, **declarative = factual, descriptive, conceptual knowledge**.

Imperative means **giving instructions, commands, or showing how to do something**.

In the context of knowledge, **imperative knowledge** refers to:

“Knowing *how* to do something.”

It is procedural knowledge — steps, methods, or processes.

Examples (general):

- Knowing how to tie a shoe
- Knowing how to follow a recipe
- Knowing how to perform a task step-by-step

So, **imperative = procedural, action-based, “how-to” knowledge**.

Example: Solving a Word Problem (Interactive Session)

Problem:

A car travels 60 km in 1 hour. How far will it travel in 4 hours at the same speed?

1. Declarative Knowledge (Knowing *what*)

This involves the **facts and concepts** you must already know:

- Distance = Speed × Time
- Speed is the rate of movement
- Hour is a unit of time
- Kilometres measure distance

These facts help you *understand* the problem.

2. Imperative Knowledge (Knowing *how*)

This involves the **steps or procedure** to **solve** the problem:

Identify the speed: 60 km/hour

Identify the time: 4 hours

Apply the formula: Distance = Speed × Time

Multiply: 60 × 4

Get the answer: 240 km

This is the **method** or **process**.

Let’s talk about ALGORITHM

1) Sequence of simple steps

2) Flow of control process that specifies when each step is executed

3) A means of determining when to stop

Question 1: Which of the two illustrations in the previous slide is an Algorithm?

ALGORITHMS are RECIPES / RECIPES are ALGORITHMS

Bake cake from a box

1) Mix dry ingredients

2) Add eggs and milk

3) Pour mixture in a pan

4) Bake at 350F for 5 minutes

5) Stick a toothpick in the cake

    6a) If toothpick does not come out clean, repeat step 4 and 5

    6b) Otherwise, take pan out of the oven

7) Eat

COMPUTERS are MACHINES that EXECUTE ALGORITHMS

Two things computers do:

- Performs simple operations 100s of billions per second!
- Remembers results 100s of gigabytes of storage!

What kinds of calculations?

- Built-in to the machine, e.g., +
- Ones that you define as the programmer

The BIG IDEA here?

A COMPUTER WILL ONLY DO WHAT YOU TELL IT TO DO.

Let’s Discuss: Do you agree?

Computer Architecture and **History**.

Fixed program: is a machine designed and built to perform **only one specific task** or a fixed set of algorithms. To change the task, the machine's physical structure must be altered.

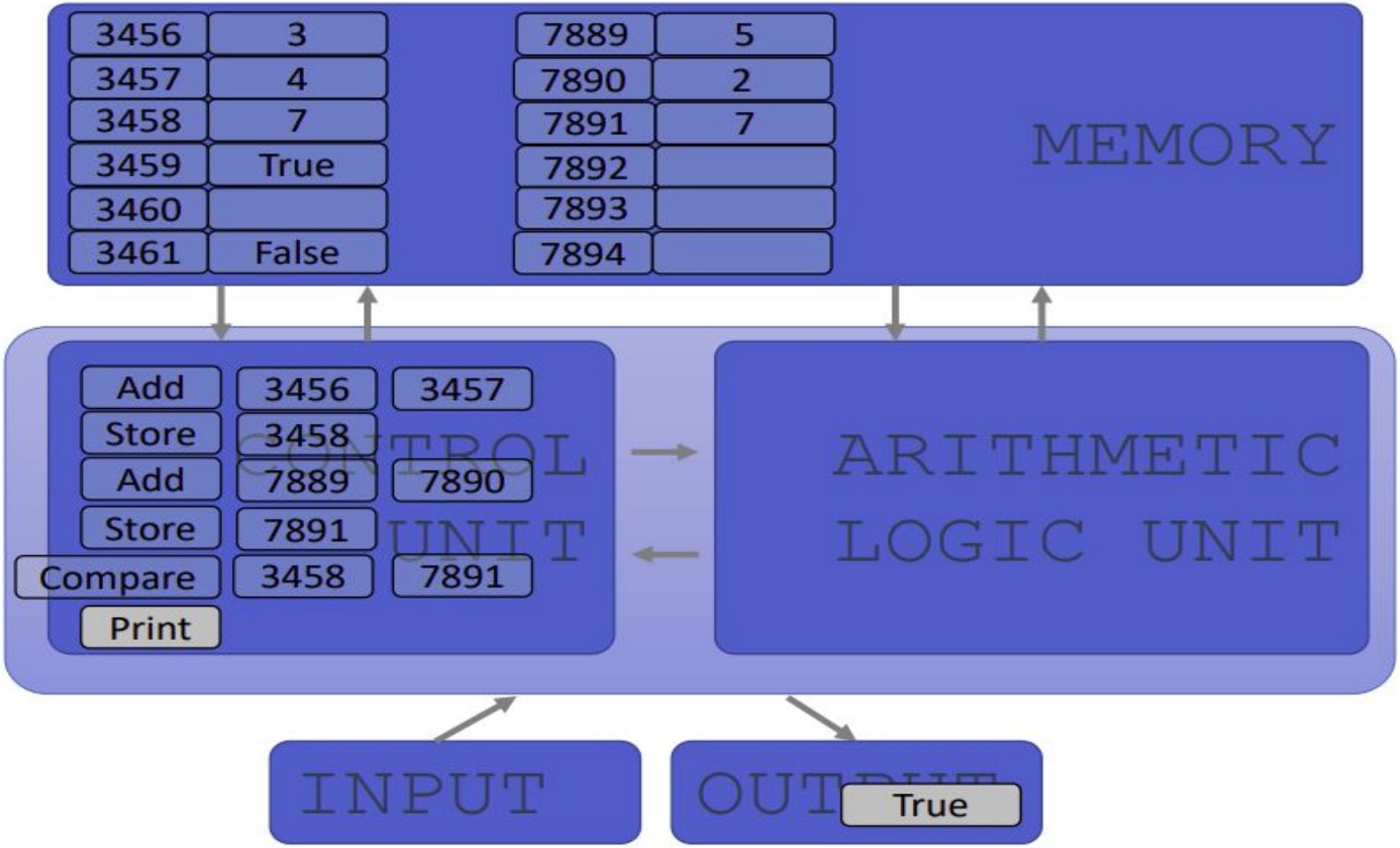
Stored program: is a machine where **both the program instructions and the data** they operate on are stored in the same high-speed, internal memory.

Stored Program Computer Explained

A **stored program computer** is a type of computer where **both data and program instructions are stored in memory**. The CPU reads and executes these instructions sequentially.

Key Features:

1. **Sequence of Instructions Stored Inside the Computer**
   - All instructions (programs) are kept in the computer's memory, just like data.
   - The computer fetches instructions one by one and executes them.
2. **Built from a Predefined Set of Primitive Instructions**
   - The CPU can only execute basic operations it understands, called **primitive instructions**:
     - **Arithmetic operations:** add, subtract, multiply, divide
     - **Logical operations:** AND, OR, NOT, comparisons
     - **Data movement:** moving data between memory and CPU
     - **Simple tests / conditions:** e.g., check if a value is zero


3456
3
3457
4
3458
7
3459
True
3460
3461
False
7889
5
7890
2
7891
7
7892
7893
7894
MEMORY
Add
3456
3457
Store
3458
Add
7889
7890
Store
7891
Compare
3458
7891
Print
CONTROL UNIT
ARITHMETIC LOGIC UNIT
INPUT
OUTPUT
True

Programming Language Aspects

Programming languages are defined by three main aspects: Primitives, Syntax, and Semantics.

Primitives

In computer science, **primitives** are the **most basic operations that a computer's CPU can perform directly**. They form the building blocks of all programs and higher-level instructions.

Categories of Primitives:

Arithmetic Primitives: Perform basic mathematical operations

- Examples: `add`, `subtract`, `multiply`, `divide`

- **Logical Primitives:** Perform simple logical operations
  - Examples: `AND`, `OR`, `NOT`, `compare`
- **Data Movement Primitives:** Move data between memory, registers, and CPU
  - Examples: `load`, `store`, `copy`
- **Test / Conditional Primitives:** Check conditions and guide program flow
  - Examples: `if zero`, `if negative`, `jump if equal`

Example:

If you want to **add two numbers stored in memory**, the CPU might execute these primitives:

1. `LOAD` number 1 into register
2. `LOAD` number 2 into another register
3. `ADD` the two registers
4. `STORE` the result back into memory

Syntax

It refers to the rules that define how statements and expressions must be written in a language so that they are correctly understood. Every language, whether natural (like English) or programming, has its own syntax rules.

1. Syntax in English

- **Not every combination of words makes a valid sentence.**
- **Example:**
  - `"cat dog boy"` → ❌ **Not syntactically valid (does not follow English grammar rules)**
  - `"cat hugs boy"` → ✅ **Syntactically valid (follows English grammar rules)**

2. Syntax in Programming Languages

- **Programming languages also have rules about how code must be written.**
- **Examples in Python:**
  - `"hi"5` → ❌ **Not syntactically valid (cannot write a string directly followed by a number)**
  - `"hi"*5` → ✅ **Syntactically valid (repeats the string 5 times)**

Static Semantics

Static semantics defines **which syntactically correct statements actually have meaning** in the language.

A statement can follow all the syntax rules but still be **meaningless or incorrect** according to the language's rules.

Static semantic checks are usually done **before the program runs** (at compile time in programming languages).

Examples in English

`"I are hungry"` → ✅ Syntactically valid (it follows basic sentence structure rules)

❌ Static semantic error: the verb "are" does not agree with the subject "I"

Correct version: `"I am hungry"`

Examples in Programming Languages

`"hi" + 5` → ✅ Syntactically valid (correct Python expression syntax)

❌ Static semantic error: you cannot add a string and an integer in Python

Correct versions:

- `"hi" * 5` → repeats the string 5 times
- `"hi" + str(5)` → concatenates `"hi"` and `"5"`

Key Points

Syntax = correct form or structure of statements

Static semantics = meaningfulness of statements based on language rules

A program must be both **syntactically correct** and **semantically valid** to execute properly

Semantics (Dynamic / Full Semantics)

- Deals with **the meaning and behavior of statements when the program actually runs** (run-time).
- Focuses on **what the program does** rather than just whether it is well-formed.
- Example in Python:
  - `x = 1 / 0` ✅ Syntactically valid and passes static semantic checks
  - ❌ Runtime semantic error (division by zero)

- Example in English:
  - `"The car drank the water"` ✅ Syntactically valid
  - ❌ Semantically odd or meaningless

Types of Programming Errors

In programming, errors can occur at different stages: **syntactic errors** and **static semantic errors**. Understanding these helps you debug effectively.

1. Syntactic Errors

- Occur when code **violates the rules of the programming language** (syntax).
- **Characteristics:**
  - Common and easy to detect.
  - Prevent the program from running.

Example:

```
print("Hello world"  # Missing closing parenthesis → syntax error
```

- **Tip:** Always read error messages; most compilers/interpreters point to the syntax error.

2. Static Semantic Errors

- Occur when code is **syntactically correct** but **violates language rules or has meaningless operations**.
- **Characteristics:**
  - Some languages can check these **before running** the program (compile-time checks).
  - Can cause **unpredictable behavior** if not detected.
  - No linguistic/syntax errors, but **the program behaves differently than intended**.

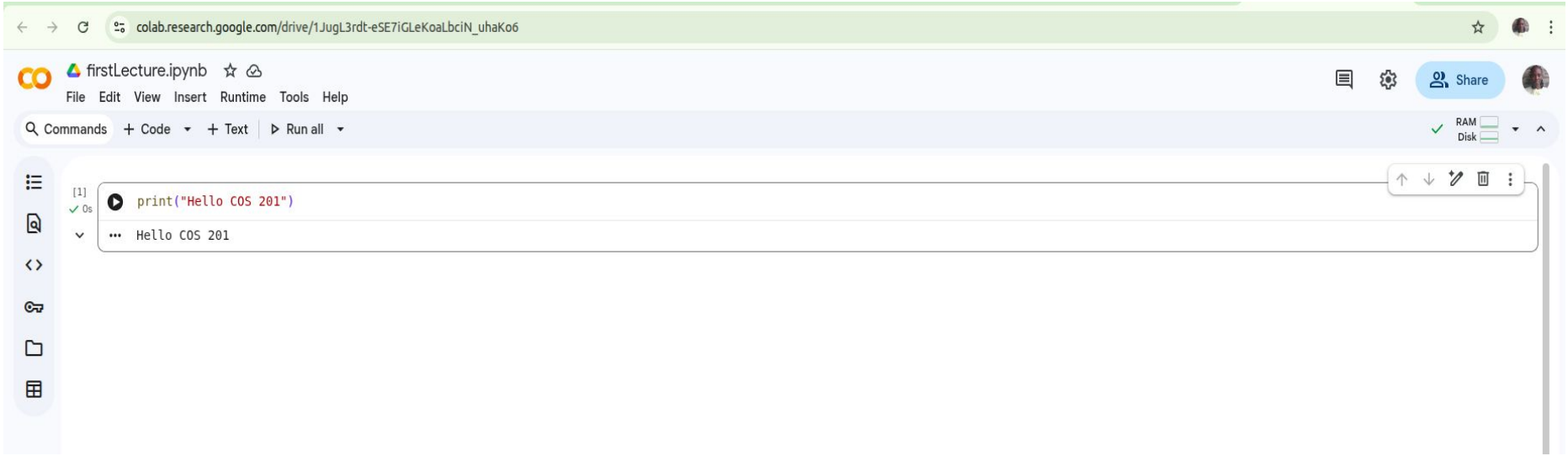
- **Examples:**
  - `"hi" + 5` → type mismatch (cannot add string and integer)
  - Using an undeclared variable

3. Runtime / Dynamic Errors

- Occur **while the program is running**.
- **Characteristics:**
  - Program may crash or stop unexpectedly.
  - Program may run forever (infinite loop).
  - Program may produce an incorrect result.
- **Examples:**
  - `1 / 0` → division by zero
  - Logic errors: wrong formula producing incorrect output

PROGRAMS

- A program is a sequence of definitions and commands
  - Definitions evaluated
  - Commands executed by Python interpreter in a shell
- Commands (statements) instruct interpreter to do something
- Can be typed directly in a shell or stored in a file that is read into the shell and evaluated


colab.research.google.com/drive/1JugL3rdt-eSE7iGLeKoaLbciN_uhaKo6
firstLecture.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands
+ Code
+ Text
Run all
RAM
Disk
[1]
0s
print("Hello COS 201")
Hello COS 201

Objects in Programming

In Python (and most programming languages), **programs manipulate objects**. Objects are **data items** that have certain **types**, and the type determines **what operations can be performed** on them.

Examples of Objects in Python

1. **Numbers (Integers and Floats)**
   - Example: `10` → integer
   - Example: `3.14` → float
   - Operations allowed: `+`, `-`, `*`, `/`, `**` (exponentiation), `%` (modulus)

2. **Strings (Sequence of Characters)**
   - Example: `'Hello'` → string
   - Example: `"Python"` → string
   - Operations allowed: concatenation (`+`), repetition (`*`), slicing (`[start:end]`)

3. Booleans (True/False Values)

- Example: `True` → boolean
- Example: `False` → boolean
- Operations allowed: logical operations (`and`, `or`, `not`)

4. Lists (Ordered Collection of Objects)

- Example: `[1, 2, 3]` → list of integers
- Example: `['a', 'b', 'c']` → list of strings
- Operations allowed: indexing (`[0]`), slicing, appending (`.append()`), removing (`.remove()`)

SCALAR OBJECTS

- `int` – represent **integers**, ex. `5, -100`
- `float` – represent **real numbers**, ex. `3.27, 2.0`
- `bool` – represent **Boolean** values `True` and `False`
- `NoneType` – **special** and has one value, `None`
- Can use `type()` to see the type of an object

```
>>> type(5)
int
>>> type(3.0)
float
```

what you write into the Python shell

what shows after hitting enter

Quiz

In your console, find the type of:

- 1234
- 8.99
- 9.0
- True
- False

TYPE CONVERSIONS (CASTING)

- Can **convert object of one type to another**
  - `float(3)` casts the int `3` to float `3.0`
  - `int(3.9)` casts (note the truncation!) the float `3.9` to int `3`
- Some operations perform implicit casts
  - `round(3.9)` returns the int `4`

Quiz 2

In your console, find the type of:

- float(123)
- round(7.9)
- float(round(7.2))
- int(7.2)
- int(7.9)

EXAMPLES

- `>>> 3+2`
- `5`
- `>>> (4+2)*6-1`
- `35`
- `>>> type((4+2)*6-1)`
- `int`
- `>>> float((4+2)*6-1)`
- `35.0`

Do computations left to right – like in math!

Do computations inside parens first, left to right

Take care about what operations you are doing

Quiz 3

In your console, find the type of:

- (13-4) / (12*12)
- type(4*3)
- type(4.0*3)
- int(1/2)

OPERATORS on `int` and `float`

- `i+j` → the **sum**
- `i-j` → the **difference**
- `i*j` → the **product**
- `i/j` → **division**

if both are ints, result is int
if either or both are floats, result is float

result is always a float

- `i//j` → **floor division**
- `i%j` → the **remainder** when `i` is divided by `j`

What is type of output?

- `i**j` → `i` to the **power** of `j`

SIMPLE OPERATIONS

- Parentheses tell Python to do these operations first
  - Like math!
- **Operator precedence** without parentheses

**	
* / %	executed left to right, as appear in expression
+ −	executed left to right, as appear in expression

VARIABLES

- Computer science variables are **different** than math variables
- **Math variables**
  - Abstract
  - Can **represent many values**

```
a + 2 = b - 1
```

```
x * x = y
```

x represents all square roots

- **CS variables**
  - Is bound to **one single value** at a given time
  - Can be bound to an expression (but expressions evaluate to one value!)

```
a = b + 1
```

```
m = 10
F = m*9.98
```

one variable

ABSTRACTING EXPRESSIONS

- Why **give names** to values of expressions?
  - To **reuse names** instead of values
  - Makes code easier to read and modify
- Choose variable names wisely
  - Code needs to read
  - Today, tomorrow, next year
  - By you and others
  - You'll be fine if you stick to letters, underscores, don't start with a number

```
#Compute approximate value for pi
pi = 355/113
radius = 2.2
area = pi*(radius**2)
circumference = pi*(radius*2)
```

comments start with a # and are not part of code executed – used to tell others what your code is doing

Expression on the right
Variable on the left

Computer Programming I Using Python